

DOCUMENTATION TECHNIQUE

Système de gestion de quincaillerie

Fiche technique — architecture, données, intégrations et
exploitation

Version du document : 1.0

Date : 19 juillet 2026

Public : équipe technique / développeurs

Document confidentiel — usage interne

Table des matières

1.	Présentation du projet	3
2.	Architecture générale	4
3.	Stack technique et dépendances	6
4.	Modèle de données	7
5.	Le noyau de stock unifié	9
6.	Machine à états des commandes	11
7.	Système de permissions dynamique	12
8.	Intégration IA (Google Gemini)	13
9.	Paieement et facturation	14
10.	Installation et configuration	15
11.	Sauvegardes et restauration	17
12.	Limites connues et évolutions prévues	18

1. Présentation du projet

1.1. Objectif

L'application est un système de gestion commerciale pour une quincaillerie, couvrant en un seul outil : la vente au comptoir (caisse/POS), la gestion de stock multi-entrepôts, les achats fournisseurs, une boutique en ligne pour les clients, et un ensemble de fonctions d'assistance basées sur l'IA (Google Gemini). L'ensemble repose sur une seule base de données partagée : le stock, les prix et les clients sont strictement identiques que la vente ait lieu au comptoir ou en ligne.

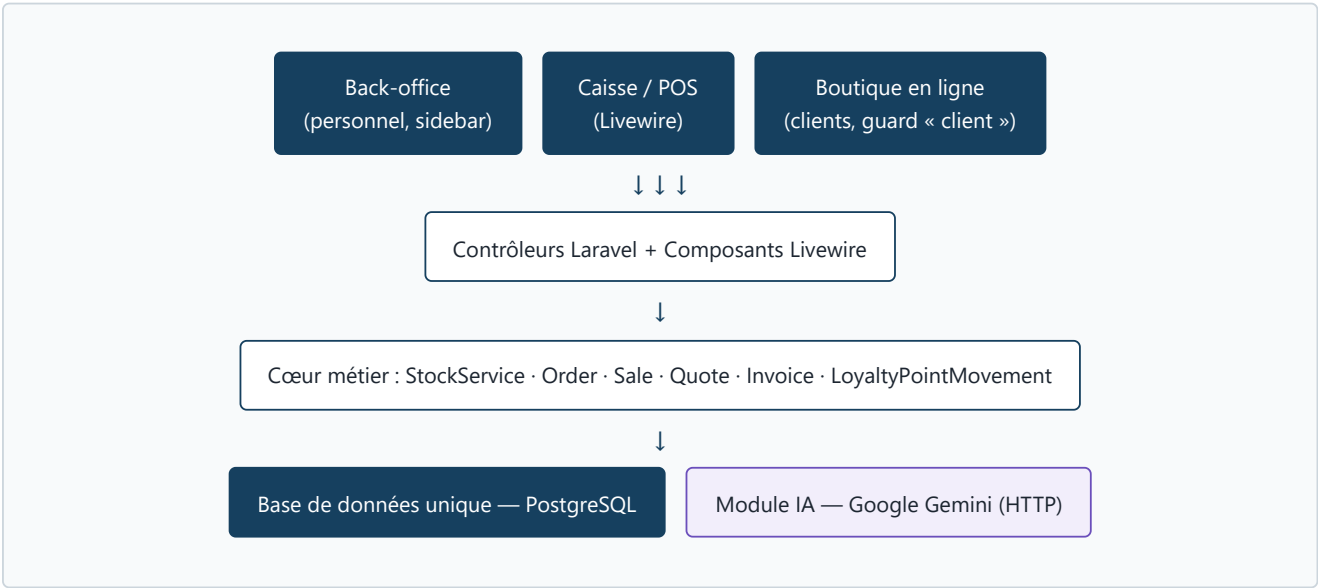
1.2. Périmètre fonctionnel réel

Ce document décrit uniquement ce qui est effectivement implémenté dans le code au moment de sa rédaction. Les fonctionnalités partiellement implémentées ou non validées sont explicitement signalées par un encadré **Attention** ou **Non implémenté**.

- Vente comptoir (caisse) avec session de caisse, découpe au mètre, tarification professionnelle, points de fidélité ;
- Boutique en ligne (catalogue public, panier, commande, compte client, notifications) ;
- Gestion de stock multi-entrepôts avec lots, dates de péremption, transferts et inventaires ;
- Achats fournisseurs (commandes, réception, calcul du CUMP) ;
- Devis convertibles en vente ou en commande réservée ;
- Service après-vente (garantie, retours, réparations) ;
- Rapports de pilotage (ventes, stock, encours clients, articles dormants, trésorerie) ;
- Assistant IA (chatbot outillé), diagnostic technique client, réapprovisionnement intelligent, détection de stock dormant ;
- Permissions dynamiques par profil, journal d'audit, journal des erreurs critiques, sauvegardes planifiables.

2. Architecture générale

L'application est une application Laravel monolithique (un seul déploiement, une seule base de données PostgreSQL). Trois façades d'utilisation partagent le même cœur métier :



Points clés de cette architecture :

- **Une seule base de données** : aucune synchronisation entre « catalogue web » et « catalogue magasin » — c'est la même table `products` , le même stock, les mêmes prix.
- **Deux systèmes d'authentification distincts** (deux guards Laravel) : `web` pour le personnel (modèle `User` , permissions Spatie) et `client` pour les clients de la boutique (modèle `Customer` , table de compte web dédiée).
- **Un seul point d'entrée pour le stock** : quel que soit le canal (comptoir ou web), toute variation de stock passe par `StockService` (voir chapitre 5) — il n'existe pas de logique de stock parallèle.
- **Le module IA est un service HTTP externe optionnel** : l'application fonctionne sans clé Gemini configurée (les fonctions IA se désactivent proprement avec un message explicite plutôt que de planter).

2.1. Découpage des dossiers principaux

Dossier	Contenu
<code>app/Http/Controllers</code>	39 contrôleurs métier + sous-dossiers <code>Auth/</code> , <code>Payment/</code> , <code>Shop/</code> (boutique en ligne)
<code>app/Models</code>	34 modèles Eloquent (catalogue, stock, ventes, achats, e-commerce, IA, audit)
<code>app/Services/Stock</code>	<code>StockService</code> — noyau de stock unifié (chapitre 5)
<code>app/Services/Ai</code>	<code>GeminiService</code> , <code>ChatbotTools</code> , <code>ReorderAdvisor</code> , <code>DormantStockAdvisor</code> , <code>AnomalyDetector</code> , <code>GeminiUsageLimiter</code>
<code>app/Services/Payment</code>	Fournisseurs de paiement (simulation et agrégateur Mobile Money) derrière une interface commune
<code>app/Services/Catalog</code>	<code>ProductImportParser</code> — import CSV/photo du catalogue
<code>app/Support</code>	<code>DatabaseLock</code> , <code>ExceptionClassifier</code> , <code>ProcessOutput</code> — utilitaires transverses de fiabilité

app/Console/Commands	Tâches planifiées : sauvegarde/restauration, expiration des lots, expiration des réservations, segmentation clients, validation de la clé Gemini
resources/views/components/pos	Terminal de caisse (composant Livewire)
resources/views/components/chatbot	Assistant IA back-office (composant Livewire)

L'application compte **188 routes** et ne comporte aucun build frontend séparé (pas de `package.json`) : les vues sont du Blade côté serveur, avec Livewire pour l'interactivité (caisse, chatbot, assistant diagnostic boutique).

3. Stack technique et dépendances

3.1. Environnement d'exécution

Composant	Version
PHP	^8.3
Laravel Framework	^13.8
Base de données	PostgreSQL (extension <code>pg_trgm</code> requise pour la recherche floue)
Livewire	^4.3
spatie/laravel-permission	^8.3
laravel/tinker	^3.0

Attention — environnement local

Sur le poste de développement actuel, le binaire `php` disponible sur le PATH est celui de XAMPP (PHP 8.1.25), incompatible avec les dépendances du projet. Le binaire à utiliser est `C:\php83\php.exe`. Toute commande `artisan` doit être lancée avec ce binaire explicite.

3.2. Dépendances de développement

`fakerphp/faker`, `laravel/pail`, `laravel/pint`, `mockery/mockery`, `nunomaduro/collision`, `phpunit/phpunit` ^12.5.12.

3.3. Services externes

Service	Rôle	Obligatoire ?
Google Gemini API	Chatbot, diagnostic technique, réapprovisionnement intelligent, détection d'anomalies, description produit, génération photo	Non — désactivable, aucun SDK officiel PHP, appels HTTP bruts
Agrégateur Mobile Money	Paieement en ligne	Non — mode « simulation » par défaut (voir chapitre 9)
Serveur mail (SMTP)	Notifications (mot de passe oublié, commande prête, alertes stock, erreurs critiques)	Recommandé en production

4. Modèle de données

34 modèles Eloquent, organisés en grands domaines. Les relations principales sont listées ci-dessous ; ce chapitre ne reproduit pas le schéma SQL complet (72 migrations) mais les tables et relations qui structurent le métier.

4.1. Catalogue et stock

Modèle	Rôle
Product	Article du catalogue : prix normal/pro, seuils d'alerte, découpe au mètre (pas/step, unité), garantie, CUMP
ProductFamily	Regroupement de variantes d'un même article (ex. couleurs/dimensions)
Category	Catégorisation du catalogue
ProductSupplier	Association produit ↔ fournisseur (prix d'achat, délai de livraison <code>lead_time_days</code>)
ProductAssociation	Ventes croisées (« souvent acheté avec »), calculées par la commande <code>app:compute-cross-sell</code>
ProductLot	Lot avec date de péremption (traçabilité FEFO pour les produits qui en ont besoin)
Warehouse	Entrepôt/point de stockage
StockMovement	Journal immuable de tout mouvement de stock physique (entrée/sortie/ajustement) — table source de vérité du stock physique
Reservation	Réservation active/consommée/libérée — source de vérité du stock réservé
StockTransfer / StockTransferLine	Transfert entre entrepôts
InventoryCount / InventoryCountLine	Inventaire physique et écarts constatés

4.2. Ventes, devis, commandes

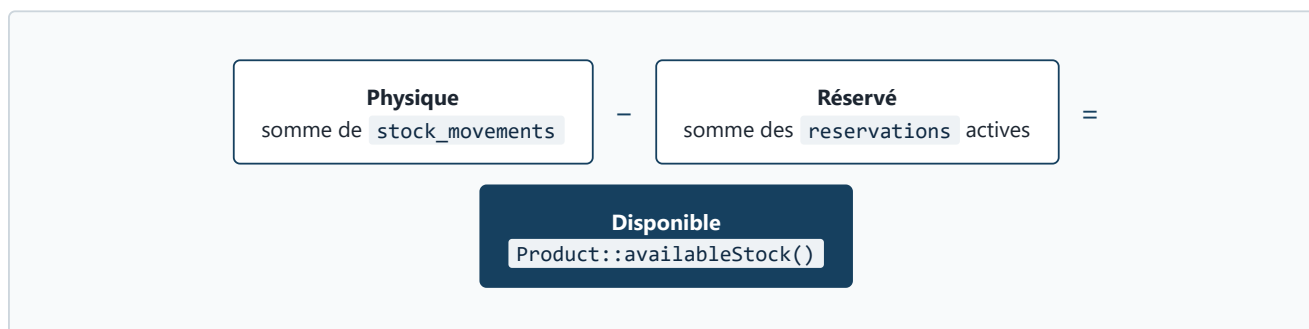
Modèle	Rôle
Sale / SaleLine	Vente comptoir ou issue d'une commande web payée
Quote / QuoteLine	Devis, convertible en vente (<code>convertToSale()</code>) ou en commande réservée (<code>convertToOrder()</code>)
Order / OrderLine	Commande en ligne — voir machine à états, chapitre 6
CashRegisterSession	Session de caisse (ouverture/clôture, fond de caisse)
Invoice / InvoiceCounter	Facture, numérotation séquentielle sans saut par année
ServiceTicket	Dossier SAV (retour, réparation, échange sous garantie)
LoyaltyPointMovement	Ledger de points de fidélité (jamais de solde stocké — toujours recalculé par somme, comme le stock)

4.3. Achats, clients, e-commerce, IA

Modèle	Rôle
PurchaseOrder / PurchaseOrderLine	Commande fournisseur, réception, retour fournisseur
Supplier	Fournisseur (délai de livraison utilisé par le réapprovisionnement intelligent)
Customer	Client (particulier ou professionnel), crédit, plafond, conditions de paiement, compte web
Payment	Paieement (comptoir, mobile money, à crédit)
ChatConversation / ChatMessage	Historique des échanges avec l'assistant IA
GeminiUsage	Compteur d'appels IA (quota journalier)
AuditLog	Journal d'audit des actions sensibles (ex. annulation de commande)
ErrorLog	Journal des erreurs critiques, avec notification aux administrateurs
User	Compte personnel, rôles/permissions Spatie

4.4. Le modèle à trois niveaux du stock

Le stock d'un produit n'est jamais un simple compteur : il se calcule à partir de deux journaux immuables.



Aucune de ces valeurs n'est stockée directement sur la table `products` : elles sont toujours recalculées depuis les journaux, ce qui élimine toute désynchronisation entre le stock affiché et le stock réel.

5. Le noyau de stock unifié

5.1. Règle absolue

Règle d'implémentation

Tout mouvement de stock (vente, commande web, retour, transfert, réception fournisseur, ajustement d'inventaire) passe obligatoirement par `App\Services\Stock\StockService`. Il n'existe aucun `StockMovement::create()` ni écriture de `Reservation` en dehors de ce service — c'est ce qui garantit qu'un verrou posé à un seul endroit protège réellement toute la contention concurrente, quel que soit le point d'entrée (comptoir, web, import, IA).

5.2. Verrouillage

Chaque opération commence par verrouiller la ligne du produit concerné (`SELECT ... FOR UPDATE`) avant toute lecture ou écriture, avec un délai d'attente borné via `DatabaseLock` (`lock_timeout` PostgreSQL, 5 secondes par défaut). Deux ventes concurrentes sur le même article ne peuvent donc jamais réussir toutes les deux sur un stock insuffisant : la seconde transaction attend que la première ait validé ou échoué avant de lire un disponible à jour — et si l'attente dépasse le délai, l'opération échoue proprement avec un message clair plutôt que de geler l'application.

5.3. Méthodes du service

Méthode	Effet	Utilisée par
<code>reserve()</code>	Réserve une quantité si disponible (rien n'est déduit physiquement)	Passage de commande web
<code>release()</code>	Libère une réservation active, idempotent	Annulation avant paiement, expiration automatique
<code>deduct()</code>	Transforme une réservation active en sortie physique réelle	Confirmation de paiement d'une commande web
<code>reserveAndDeduct()</code>	Réserve puis déduit dans la même transaction	Vente comptoir (paiement et remise simultanés)
<code>reintegrate()</code>	Réintègre du stock physique	Retour client, annulation après paiement, retour de commande
<code>withdraw()</code>	Sort du stock physique après vérification de disponibilité	Transfert, retour fournisseur
<code>adjust()</code>	Mouvement signé sans contrôle de disponibilité	Écart d'inventaire, correction manuelle
<code>receivePurchase()</code>	Verrouille, recalcule le CUMP, puis écrit l'entrée sous le même verrou	Réception d'une commande fournisseur

5.4. Origine de cette architecture — bug corrigé

Cette architecture (verrou borné + point d'entrée unique) fait suite au diagnostic d'un bug de figeage complet de l'application lors de l'annulation d'une commande en ligne et lors d'un retour partiel de vente. Cause racine identifiée : PostgreSQL n'avait aucun `lock_timeout` / `statement_timeout` configuré, et `php artisan serve` tourne avec un seul worker par défaut — une transaction bloquée sur un verrou gelait donc l'application entière pour tous les utilisateurs, pas seulement celui qui avait initié l'opération. La correction combine : un délai d'attente borné sur tous les verrous

(`DatabaseLock`), une garantie de COMMIT/ROLLBACK sur chaque opération de stock, et un verrouillage de la ligne parente (commande/vente) avant relecture de son statut pour empêcher un double traitement en cas de double clic ou de rejeu (idempotence).

6. Machine à états des commandes

Une commande en ligne (`Order`) suit un graphe de transitions strict ; toute transition hors de ce graphe est rejetée par `assertCanTransitionTo()`.



« annulée » est atteignable depuis réservée, payée, préparation et prête.

État	Effet sur le stock
reservee	Stock réservé (rien de physique) — <code>Order::place()</code>
payee	Chaque réservation active est déduite physiquement, une <code>Sale</code> est créée en parallèle, les points de fidélité sont crédités — <code>confirmPayment()</code>
annulee	Si annulée avant paiement : réservations simplement libérées. Si après paiement : stock réintégré et la <code>Sale</code> liée est marquée annulée (avec synchronisation des <code>returned_quantity</code> pour empêcher un double retour depuis la fiche vente) — <code>cancel()</code>
retournee	Stock réintégré pour la quantité non déjà retournée — <code>returnOrder()</code>

Idempotence

`cancel()` et `returnOrder()` verrouillent la ligne `Order` (`lockForUpdate`) avant de relire son statut. Deux annulations concurrentes de la même commande (double clic, deux onglets) ne peuvent donc jamais réintégrer le stock deux fois : la seconde est rejetée une fois qu'elle relit le statut à jour.

7. Système de permissions dynamique

Basé sur `spatie/laravel-permission`. Les permissions sont déclarées dans `config/permissions.php`, qui sert à la fois de source de vérité pour le seeder (`RoleSeeder`) et pour l'écran d'attribution de profils (grille de cases à cocher groupées par module). Attribuer une permission reste une opération sur les données (tables `permissions` / `model_has_permissions` / `role_has_permissions`) — ce fichier ne fait que déclarer les actions que le code sait vérifier.

15 modules, 35 permissions au total : produits, prix, e-commerce, catalogue, fournisseurs, stock, achats, ventes, caisse, clients, SAV, IA, rapports, utilisateurs, configuration système.

Module	Permissions
Produits	voir, créer, modifier, supprimer, importer (CSV/photo/OCR)
Prix	modifier (achat, vente, tarif pro)
E-commerce	publier/dépublier, gérer les commandes en ligne
Catalogue	voir, gérer catégories/familles
Fournisseurs	voir, gérer
Stock	voir, ajuster, inventaire, transférer
Achats	voir, créer/réceptionner/gérer
Ventes comptoir	créer, annuler, historique personnel, historique complet
Caisse	encaisser, ouvrir/clôturer une session, voir le journal
Clients	voir, gérer / encaisser un paiement
SAV / garantie	ouvrir et résoudre les dossiers
Intelligence artificielle	utiliser l'assistant, voir les prévisions de trésorerie
Rapports	voir, exporter les données comptables
Utilisateurs	créer/gérer les comptes, gérer les profils
Configuration système	entrepôts, journal d'audit, corbeille

Chaque entrée du menu latéral (`sidebar-nav.blade.php`) est conditionnée par une ou plusieurs de ces permissions : un utilisateur ne voit que ce à quoi il a accès, et les contrôleurs vérifient indépendamment la permission (le menu n'est donc pas la seule barrière de sécurité).

8. Intégration IA (Google Gemini)

8.1. Client Gemini

`App\Services\Ai\GeminiService` appelle l'API Gemini en HTTP brut (façade `Http` de Laravel) : il n'existe pas de SDK PHP officiel pour Gemini. L'interface publique (`chat()`, `generateText()`, `extractStructured()`, `streamChat()`) reprend volontairement le format de blocs de contenu « façon Claude » (text/image/document) utilisé par l'ancien service, pour ne pas avoir à modifier les appelants lors de la bascule.

8.2. Fonctions IA réellement implémentées

Fonction	Composant	Description
Chatbot back-office outillé	<code>ChatbotTools</code> + composant Livewire	Peut interroger le catalogue réel (recherche, stock, prix, produits en stock bas) — jamais de donnée inventée par le modèle
Assistant diagnostic boutique	<code>ChatbotTools::publicDefinitions()</code>	Sous-ensemble public sans compte requis (recherche produit, stock, prix) — exclut volontairement les données de réapprovisionnement, sensibles
Réapprovisionnement intelligent	<code>ReorderAdvisor</code>	Classement statistique (vélocité de vente, délai fournisseur <code>lead_time_days</code> , saisonnalité) + résumé narratif Gemini optionnel
Détection de stock dormant	<code>DormantStockAdvisor</code>	Classement par capital immobilisé + jours depuis la dernière vente, résumé narratif optionnel
Détection d'anomalies	<code>AnomalyDetector</code>	Signale des mouvements ou ventes atypiques
Description produit automatique	<code>ProductDescriptionController</code>	Génère une description à partir des attributs du produit
Reconnaissance / import photo	<code>ProductRecognitionController</code> , <code>ProductImportParser</code>	Extraction structurée (vision) pour préremplir une fiche produit

8.3. Gestion de la clé et du quota

- Clé configurée via `GEMINI_API_KEY` (aucune valeur par défaut) — en son absence, chaque fonction IA se désactive proprement avec un message explicite plutôt que de lever une exception non gérée.
- Modèle configurable via `GEMINI_MODEL` (par défaut `gemini-2.5-flash`).
- Quota d'appels journalier configurable via `GEMINI_DAILY_CALL_LIMIT` (0 = illimité), appliqué par `GeminiUsageLimiter` avant chaque appel.
- Commande `php artisan app:validate-gemini-key` pour vérifier la validité de la clé avant mise en production.
- Chaque échec (clé invalide, quota Google dépassé, timeout réseau) est catégorisé et traduit en message français actionnable (`lastErrorReason()` / `lastErrorMessage()`) plutôt que de remonter une erreur technique brute à l'utilisateur.

Point d'attention connu

Les modèles Gemini récents consomment une partie du budget de tokens en « réflexion » interne avant de produire le texte visible. Un budget trop court (observé à 500 tokens) peut être entièrement absorbé par cette réflexion et renvoyer un texte tronqué sans erreur HTTP. Le budget par défaut a été relevé à 2048 tokens pour les résumés narratifs.

9. Paiement et facturation

9.1. Paiement (Mobile Money)

Le paiement en ligne repose sur une interface commune (`PaymentProviderContract`) avec deux implémentations :

Fournisseur	Statut
<code>SimulatedPaymentProvider</code>	Mode par défaut (<code>PAYMENT_MODE=simulation</code>) — simule un paiement réussi/échoué sans appel réseau, utile en développement et en démonstration
<code>AggregatorPaymentProvider</code>	Squelette générique pour un agrégateur Mobile Money (CinetPay, NotchPay, PawaPay, Semoa...)

Non validé — squelette générique

Aucun agrégateur n'a été précisé/contractualisé au moment de l'écriture de ce code. La forme exacte des requêtes/réponses de `AggregatorPaymentProvider` est une hypothèse raisonnable (proche de ce que proposent la plupart de ces services),

pas une intégration vérifiée

. Avant toute mise en production avec paiement réel, ce provider doit être adapté et testé contre la documentation officielle de l'agrégateur réellement choisi (URL de base, format des requêtes, secret de webhook via `PAYMENT_AGGREGATOR_*`).

9.2. Facturation

`Invoice::generateFor()` génère (ou retrouve si déjà fait) la facture d'une vente, avec une numérotation séquentielle sans saut au format `FAC-{année}-{numéro sur 6 chiffres}`, garantie par un compteur verrouillé (`InvoiceCounter`) remis à zéro chaque année. Le verrou est borné par `DatabaseLock` pour la même raison qu'au chapitre 5 : ce compteur est le point de contention le plus probable de toute la facturation.

Non validé — mentions légales Cameroun

La numérotation séquentielle sans saut est garantie. En revanche, les mentions légales (NIU/RCCM de l'entreprise), la conformité TVA et l'intégration à une solution homologuée DGI

ne sont pas validées

. Tant que `COMPANY_NIU` / `COMPANY_RCCM` ne sont pas renseignés, la facture affiche un bandeau « non conforme ». Ne jamais présenter ces factures comme conformes sans validation par un comptable ou la DGI.

Configuration légale/fiscale dans `config/company.php` : nom, NIU, RCCM, adresse, téléphone, assujettissement TVA (`COMPANY_VAT_SUBJECT`) et taux (`COMPANY_VAT_RATE` , 19,25 % par défaut — taux standard camerounais).

10. Installation et configuration

10.1. Prérequis

- PHP 8.3 avec les extensions standard Laravel (pdo_pgsql, mbstring, openssl, ctype, fileinfo...)
- PostgreSQL avec l'extension `pg_trgm` activée (`CREATE EXTENSION pg_trgm;`) — nécessaire à la recherche floue tolérante aux fautes de frappe
- Composer
- Un compte Google AI Studio pour une clé Gemini (optionnel)

10.2. Installation

- 1 Cloner le dépôt puis installer les dépendances : `composer install`
- 2 Copier `.env.example` vers `.env` et générer la clé applicative : `php artisan key:generate`
- 3 Renseigner la connexion PostgreSQL (`DB_CONNECTION=pgsql` , `DB_HOST` , `DB_PORT` , `DB_DATABASE` , `DB_USERNAME` , `DB_PASSWORD`)
- 4 Exécuter les migrations et le seeder de rôles/permissions : `php artisan migrate --seed`
- 5 (Optionnel) renseigner `GEMINI_API_KEY` puis vérifier : `php artisan app:validate-gemini-key`
- 6 Démarrer : `php artisan serve`

10.3. Variables d'environnement principales

Écart constaté

Plusieurs variables utilisées par l'application (`GEMINI_API_KEY` , les variables `COMPANY_*` , `LOYALTY_*` , `PAYMENT_*` , les chemins `PG_*_BINARY`) existent dans le code (`config/*.php`) mais ne sont pas encore listées dans `.env.example` . Ce tableau fait référence, `.env.example` devrait être complété en conséquence.

Variable	Rôle	Défaut
<code>DB_CONNECTION</code>	Doit être <code>pgsql</code>	<code>sqlite</code>
<code>GEMINI_API_KEY</code>	Clé API Gemini	vide (IA désactivée)
<code>GEMINI_MODEL</code>	Modèle Gemini utilisé	<code>gemini-2.5-flash</code>
<code>GEMINI_DAILY_CALL_LIMIT</code>	Quota d'appels IA par jour	0 (illimité)
<code>PAYMENT_MODE</code>	<code>simulation</code> ou <code>aggregator</code>	<code>simulation</code>
<code>PAYMENT_AGGREGATOR_BASE_URL</code> / <code>_API_KEY</code> / <code>_WEBHOOK_SECRET</code>	Identifiants de l'agrégateur Mobile Money réel	vide
<code>COMPANY_NAME</code> / <code>_NIU</code> / <code>_RCCM</code> / <code>_ADDRESS</code> / <code>_PHONE</code>	Identité légale sur les factures	vide
<code>COMPANY_VAT_SUBJECT</code> / <code>COMPANY_VAT_RATE</code>	Assujettissement et taux de TVA	<code>false</code> / 19.25
<code>LOYALTY_ENABLED</code> / <code>_EARN_PER_FCFA</code> / <code>_REDEEM_VALUE</code>	Programme de fidélité	<code>true</code> / 1000 / 10

PG_DUMP_BINARY / PG_RESTORE_BINARY / PG_CREATEDB_BINARY / PG_DROPDB_BINARY / PG_PSQL_BINARY	Chemins des binaires PostgreSQL (utile si absents du PATH, cas de ce poste)	pg_dump , etc. (nom nu, cherché sur le PATH)
BACKUP_KEEP / BACKUP_TIMEOUT	Rétention et délai des sauvegardes	14 jours / 300 s

10.4. Déploiement

Aucune configuration de déploiement (CI/CD, Docker) n'est présente dans le dépôt à ce stade — c'est une évolution possible (voir chapitre 12). En production, il est recommandé de servir l'application via PHP-FPM + Nginx/Apache (plutôt que php artisan serve, mono-worker et donc plus sensible à un blocage), de configurer lock_timeout / statement_timeout au niveau PostgreSQL, et de renseigner un serveur SMTP réel pour les notifications.

11. Sauvegardes et restauration

Commandes `php artisan app:backup-database` et `php artisan app:restore-database-backup` (dossier `app/Console/Commands`), enveloppes autour de `pg_dump` / `pg_restore`, configurables via `config/backup.php`.

- Rétention configurable (`BACKUP_KEEP`, 14 par défaut) — les sauvegardes les plus anciennes sont purgées automatiquement.
- Délai d'exécution borné (`BACKUP_TIMEOUT`, 300 s par défaut) pour éviter qu'une sauvegarde bloquée ne reste indéfiniment active.
- Chemins des binaires PostgreSQL configurables (utile quand ils ne sont pas sur le PATH du système, comme sur ce poste de développement).

Restauration testée

La procédure de restauration a été effectivement vérifiée (et pas seulement écrite) par une restauration réelle suivie d'une vérification du schéma résultant. Le détail de cette vérification est documenté dans `docs/sauvegardes.md`.

12. Limites connues et évolutions prévues

12.1. Limites connues (non résolues à ce jour)

Sujet	Limite
Païement Mobile Money	<code>AggregatorPaymentProvider</code> est un squelette générique non branché sur un agrégateur réel précis (chapitre 9.1)
Facturation légale	Conformité TVA/DGI non validée par un comptable (chapitre 9.2)
Export comptable	Non validé par un comptable — bandeau d'avertissement affiché sur la page
<code>.env.example</code>	Incomplet par rapport aux variables réellement utilisées (chapitre 10.3)
Déploiement	Pas de pipeline CI/CD ni de conteneurisation fournie dans le dépôt
Serveur de développement	<code>php artisan serve</code> est mono-worker ; une opération lente bloque tous les utilisateurs tant qu'elle n'a pas atteint son <code>lock_timeout</code>

12.2. Explicitement hors périmètre (non commencé, à valider avant tout démarrage)

- Multi-magasins (l'application gère plusieurs entrepôts, pas plusieurs sociétés/points de vente indépendants)
- Tournées de livraison / logistique de transport
- Application mobile dédiée
- Comptabilité complète SYSCOHADA

12.3. Évolutions envisageables (non implémentées)

- Compléter `.env.example` avec toutes les variables réellement consommées par `config/*.php`
- Intégration réelle et testée d'un agrégateur Mobile Money précis
- Validation comptable/DGI formelle du format de facture et de l'export comptable
- Pipeline de déploiement (CI/CD, conteneurisation)